

High-Level Design

RAG-Based Customer Support Assistant

System Architecture & Component Design

Author	Himanshu Shekhar Singh
Batch	February 2026
Registration No.	IN226076602
Document Type	High-Level Design (HLD)

1. System Overview

Customer support systems often rely on static FAQs or human agents, which can lead to delayed responses, high operational costs, and limited scalability. This project addresses these limitations by building an intelligent assistant using Retrieval-Augmented Generation (RAG).

System Capabilities

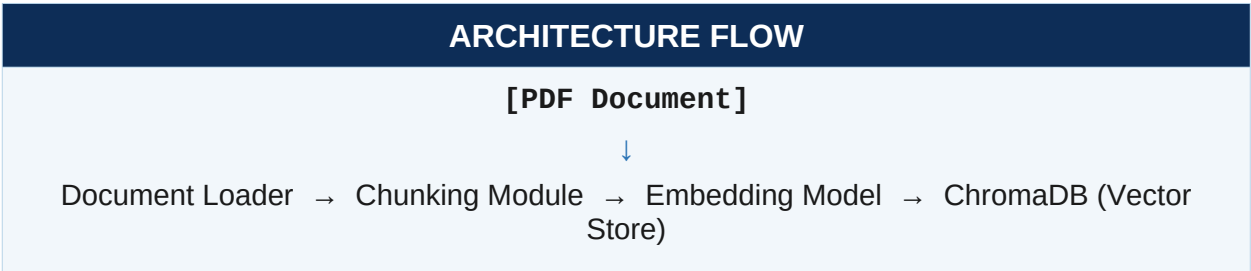
- Understands and processes user queries in natural language
- Retrieves relevant information from a PDF knowledge base
- Generates accurate, contextual responses using an LLM
- Escalates complex queries to human agents when necessary

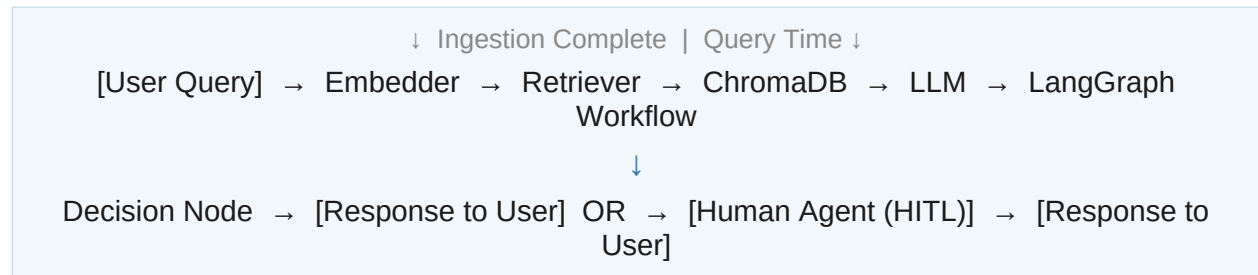
System Scope

The System WILL	The System WILL NOT
✓ Process and store PDF knowledge base	✗ Support real-time database updates
✓ Use embeddings for semantic search	✗ Handle multilingual queries (optional)
✓ Retrieve relevant document chunks	✗ Replace human agents completely
✓ Generate responses using an LLM	
✓ Use graph-based workflow via LangGraph	
✓ Implement HITL escalation	

2. Architecture Overview

The architecture follows a two-phase design: a document ingestion pipeline and a query processing pipeline, orchestrated through a LangGraph workflow engine.





3. Component Descriptions

Component	Description
Document Loader	Reads and extracts text from the input PDF file, converting unstructured documents into machine-readable format for further processing.
Chunking Module	Divides extracted text into smaller, manageable segments to improve retrieval accuracy and ensure only relevant portions are processed.
Embedding Model	Converts text chunks into numerical vector representations, capturing semantic meaning to enable similarity-based retrieval.
Vector Store (ChromaDB)	Stores embeddings efficiently with fast similarity search capabilities. Chosen for its simplicity, lightweight nature, and Python integration.
Retriever	Fetches the most relevant chunks from the vector database using cosine similarity search, providing context for the LLM.
LLM	Processes retrieved context along with the user query to generate meaningful, context-aware responses in natural language.
Graph Workflow Engine (LangGraph)	Manages the flow of operations in a graph-based structure, enabling conditional routing and controlled execution order.
Routing / Decision Layer	Evaluates the generated response based on confidence level, completeness, and query complexity; routes to user or escalates.
HITL Module	Handles escalation to a human agent when the system cannot provide a reliable answer; delivers the human response back to the user.

4. Data Flow

a) Document Ingestion Flow

The system begins by processing the input PDF document. The document loader extracts text content, which is then divided into smaller chunks by the chunking module. These chunks are passed to the embedding model, which converts them into vector representations. The generated embeddings are stored in ChromaDB for efficient retrieval during query time.

b) Query Processing Flow

When a user submits a query, it is converted into an embedding and passed to the retriever. The retriever searches the vector database and fetches the most relevant chunks based on similarity. These retrieved chunks are provided as context to the LLM, which generates a response. The response is then passed to the LangGraph workflow engine.

c) Decision and Response Flow

The generated response is evaluated by the decision layer. If the response meets confidence and relevance criteria, it is returned to the user. If the system detects low confidence, missing context, or complex queries, the request is escalated to a human agent via the HITL module.

5. Technical Choices

Technology	Choice	Rationale
Vector Database	ChromaDB	Simple, lightweight, Python-native. Well-suited for small to medium-scale applications with efficient similarity search.
Workflow Orchestration	LangGraph	Graph-based execution with conditional routing; superior to linear pipelines for complex decision-making.
Embedding Model	Semantic Embeddings	Context-based retrieval improves accuracy over keyword matching; captures meaning rather than exact words.
LLM	Large Language Model	Enables natural language understanding and contextual answer generation for human-like responses.
Programming Language	Python	Extensive AI/ML ecosystem; strong library support for all project components.

6. Scalability Considerations

Concern	Mitigation Strategy
Large Documents	Dynamic chunk sizing and batching for efficient processing.
High Query Load	Parallel processing and load balancing; cloud infrastructure deployment.
Vector DB Performance	Indexing techniques and approximate nearest neighbor (ANN) search for faster retrieval.
Response Latency	Cache frequently asked queries; optimize top-k tuning and prompt size.
HITL at Scale	Queue-based system for managing escalated queries efficiently.